

stress-strike

To'liq O'rganish Qo'llanmasi (Study Guide)

Go load-testing va security suite — arxitektura, oqimlar va mustaqil o'rganish rejasi

Mundarija

0-daraja: Loyiha bir paragrafda

1-daraja: Asosiy tushunchalar (portfolioda suhbat uchun yetarli)

2-daraja: Papka tuzilmasi — "xarita"

Har bir paket — bitta gap bilan

3-daraja: Asosiy oqimlar (asl ish qanday ketadi)

Oqim A: CLI ishga tushishi

Oqim B: cmdRun — load test oqimi

Oqim C: engine.Run — yadro ishi (eng muhim oqim!)

Oqim D: Natasha o'lchash → hisobot

4-daraja: Paketlar bo'yicha chuqur (kerak bo'lganda)

internal/config — scenario svetofori

internal/engine — yadro

internal/metrics

internal/report

internal/replay

internal/scanner — xavfsizlik

internal/dashboard

internal/dist (distributed)

api

Qaysi mavzularni mustaqil o'rganish kerak (Go asoslari)

Muammoli joylar namunalari (suhbatda gapirish uchun)

Tekshiruv savollari (o'z-o'zini baholash)

Xulosa

stress-strike — To'liq O'rganish Qo'llanmasi (Study Guide)

Bu hujjat **stress-strike** loyihasini **tushunib olish** uchun tuzilgan.

Maqsad — butun kodni yodlash emas, balki **katta rasmni** va har bir

bo'limning vazifasini tushunib olish.

Qanday o'qish kerak: 1-daraja (asos) → 2-daraja (xarita) → 3-daraja

(oqimlar) → 4-daraja (chuqur, ixtiyoriy). Har bir darajani o'z so'zingiz

bilan tushuntira olsangiz — keyingisiga o'ting.

0-daraja: Loyiha bir paragrafda

stress-strike — Go tilida yozilgan, ikki maqsadli vosita:

1. **Yuklama (load) testi** — ko'p virtual foydalanuvchi bir vaqtda HTTP/gRPC/WebSocket so'rovlarini yuboradi va sayt/API qancha yuk ko'tarishini o'lchaydi.

1. **Xavfsizlik (security) skaneri** — TLS/WAF/O'zboshimchalik (vulnerability) zaifliklarni tekshiradi va pentest hisobotini chiqaradi.

Bundan tashqari: **traffic replay** (PCAP/HAR real trafikni qayta ijro etish),

distributed mode (master/worker orqali ko'p mashinada yuklash),

real-vaqt dashboard, **interactive wizard** bor.

Nima uchun Go? — bir vaqtda ko'p so'rov yuborish uchun yuqori konkurentlik (goroutine), kompilyatsiya qilingan binary (tezkor), oddiy deploy kerak.

1-daraja: Asosiy tushunchalar (portfolioda suhbat uchun yetarli)

Agar buni o'z so'zing bilan ayta olsangiz — fundament tayyor:

1. **Virtual user (VU)** — testda bitta "foydalanuvchi" simulyatsiyasi; har biri o'z goroutine'ida so'rov yuboradi.

1. **Load profile** — foydalanuvchilar soni vaqt o'tishi bilan qanday o'zgarishi (doimiy / asta oshib boruvchi / spike / to'lqin).

1. **RPS (Requests Per Second)** — bir soniyada yuborilgan so'rovlar soni (asosiy "tezlik" ko'rsatkichi).

1. **Latency (kutinish)** — so'rov yuborilgandan javob kelguncha vaqt.

2. **Percentile (P50/P95/P99)** — hujjatlar foizini ifodalaydigan o'lchov: P99 = so'rovlarning 99% shu qiymatdan tezroq.

1. **SLA gate** — test oxirida "natija me'yordan o'tdimi?" degan tekshiruv (CI/CD'da exit code 2 bilan fail qiladi).

1. **Assertion** — javob tanasida kutilgan shartni tekshirish (masalan `status == 200`).

1. **Scenario** — testni tavsiflovchi sozlashlar to'plami (YAML fayl yoki flag).

2-daraja: Papka tuzilmasi — "xarita"

Reponing asosiy qismlari:

```
stress-strike/
■■■ api/                # Qayta foydalanish mumkin bo'lgan yuqori darajadagi API
■■■ cmd/                # Barcha bajariladigan dasturlar (binary entry points)
■   ■■■ stress-strike/  # Asosiy CLI (run, replay, scan, pentest, ...)
■   ■■■ stress-strike-dashboard/
■   ■■■ stress-strike-replay/
■   ■■■ stress-strike-scan/
■   ■■■ stress-strike-master/
■   ■■■ stress-strike-worker/
■■■ examples/          # YAML scenario namunalari + demo server
■■■ internal/           # Ichki paketlar (boshqa loyihalarga export qilinmaydi)
■   ■■■ config/         # Scenario yuklash/validatsiya
■   ■■■ engine/         # Asosiy yuklama mashinasi (yadro)
■   ■■■ metrics/        # Natijalarni o'lchash (histogram, telemetry, timeline)
■   ■■■ report/         # Hisobot chiqarish (terminal, JSON, HTML, PDF)
■   ■■■ replay/         # PCAP/HAR trafikni qayta ijro etish
■   ■■■ scanner/        # Xavfsizlik skaneri (TLS, WAF, CVE, OWASP, auth)
■   ■■■ dashboard/      # Real-vaqt web dashboard server
■   ■■■ dist/           # Distributed (master/worker) gRPC protokoli
■■■ scripts/           # install.sh, build-all.sh, release.sh
■■■ docs/              # Hujjatlar (shu hujjat)
■■■ Makefile            # Build/test/lint/release buyruqlari
■■■ Dockerfile
■■■ go.mod              # Modul: github.com/asadbekabdulboqiyev/stress-strike
```

Har bir paket — bitta gap bilan

| Paket | Vazifasi |

|---|---|

| **cmd/stress-strike** | Asosiy CLI — flag'lar, subcomanda tanlash, buyruq chaqirish |

| **internal/config** | YAML/JSON scenario ni yuklab, to'g'rilab (normalize), validatsiya qiladi |

| **internal/engine** | Testni **haqiqatda** ijro etadi — userlar, profil, so'rov yuborish |

| **internal/metrics** | Barcha o'lchovlarni yig'adi: histogram, percentile, telemetry, timeline |

| **internal/report** | Natijalarni chiroyli hisobotga aylantiradi (terminal/JSON/HTML/PDF) |

| **internal/replay** | Real PCAP/HAR trafikni o'qib, qayta yuklama qiladi |

| **internal/scanner** | Xavfsizlik tekshiruvlari: TLS, WAF, HTTP server bilgilar, CVE, OWASP |

| **internal/dashboard** | Brauzerda real-vaqt ko'rsatuvchi web server |

| **internal/dist** | Master/worker ko'p-mashina ishlash protokoli (gRPC) |

| **api** | Oddiy, qayta foydalanadigan **Run(ctx, cfg)** API |

3-daraja: Asosiy oqimlar (asl ish qanday ketadi)

Bu — loyihaning eng muhim qismi. Bir vaqtni o'tkazib, har bir oqimni yakka o'zingiz kuzatib boring.

Oqim A: CLI ishga tushishi

```
go run ./cmd/stress-strike <buyruq>
  ▼
main.go main() ■■ os.Args[1] ga qarang
  ■ "version"/"help" → chop et va chiq
  ■ "run"            → cmdRun()
  ■ "replay"         → cmdReplay() → runSubCommand("stress-strike-replay")
  ■ "scan"           → cmdScan()
  ■ "dashboard"      → cmdDashboard()
  ■ "master"/"worker" → distributed
  ■ "pentest"        → cmdPentest()
  ■
```

■■■■ hech narsa bo'lsa: TTY bo'lsa → interactivePicker(), aks holda cmdRun()

Muhim: `run` asosiy buyruqdir. `replay/scan/dashboard/master/worker`

alohida binary'larni (`cmd/` ostidagi) chaqiradi — `runSubCommand()` orqali.

Oqim B: `cmdRun` — load test oqimi

```
cmdRun()
  ■■■■ flag'lar e'lon qilindi (--url, --users, --duration, --profile, ...)
  ■■■■ Scenario yaratildi:
  ■     configPath berilsa → config.Load(path)    (YAML fayl)
  ■     --url berilsa      → quickScenario(...)   (flaglardan tez scenario)
  ■■■■ SLA/AQL? scenario.SLA ni o'qing
  ■■■■ engine.New(scenario) → Engine obyekti
  ■■■■ eng.Run(ctx, RunOptions{...}) → <--- test SHU yerda ishlaydi
  ■■■■ natija report.Build(...) bilan Report ga aylantirildi
  ■■■■ terminal report chop etildi (display)
  ■■■■ SLA tekshirildi → exit code
  ■■■■ JSON/TXT fayllar saqlandi
```

Oqim C: `engine.Run` — yadro ishi (eng muhim oqim!)

Bu ichida quyidagilar bo'ladi:

```
Engine.Run(ctx, opts)
  ■■■■ LoadProfile yaratildi (steady/ramp/spike/wave/constant-rps)
  ■■■■ pre-warm (so'ralgan bo'lsa): oldindan TCP ulanishlar ochildi
  ■■■■ har bir virtual user uchun goroutine ishga tushdi
  ■     ■■■■ har bir user o'z "step"larni (config.Step) bajaradi:
  ■         HTTP/GPRC/WebSocket/raw TCP so'rov yuboradi
  ■■■■ har bir so'rov natijasi o'lchandi → metrics.Telemetry
  ■■■■ progress tracker ishladi (---% live)
  ■■■■ capture (so'ralgan bo'lsa): muvaffaqiyatsiz javoblar saqlandi
  ■■■■ pooling (so'ralgan bo'lsa): payload pool'dan ma'lumot olinadi
  ■■■■ gate (so'ralgan bo'lsa): race-shot "bir vaqtda" sinov
  ■■■■ yakunda natija qaytdi → report paketi
```

So'rov yuborish `internal/engine` da quyidagi "client" fayllari orqali:

`client.go` (HTTP), `grpc_client.go`, `ws_client.go` (WebSocket),
`raw_client.go` (TCP), `streaming.go`.

Oqim D: Natasha o'lchash → hisobot

```
engine → metrics.Telemetry
  ■■■■ Histogram (latency taqsimoti)
  ■■■■ status code hisobi
  ■■■■ xato (error) type hisobi
  ■■■■ timeline (har soniya namunasi)
  ▼
report.Build(t, scenario) → Report struct
  ■■■■ report.Compare (baseline bilan solishtirish)
  ■■■■ report.EvaluateSLA
  ■■■■ display (terminalda chiroyli panel)
```

4-daraja: Paketlar bo'yicha chuqur (kerak bo'lganda)

`internal/config` — scenario svetofori

Asosiy tiplar:

- **Scenario** — loyihaning "pasporti": `Name`, `BaseURL`, `Profile`, `Steps`, `Variables`, `SLA`, `PreWarm`.
- **Profile** — yuklama shakli: `Type`, `Users`, `Duration`, `Warmup`, `RampUp`, `Spike*`, `WavePeriod`, `RPS`, `TargetRPS`, `Gate`, `RateLimit`.
- **Step** — bitta so'rov qadami: `Name`, `Method`, `URL`, `Body`, `Headers`,

Assertions, Extract.

- **SLA** — me'yorlar: max P99, max error rate, min RPS.
- **Assertion** — javob tanasi tekshiruvi.
- **RateLimitConfig** — global tezlik cheklovi (token bucket).

Vazifalar:

- **Load(path)** — YAML/JSON o'qiydi
- **LoadJSON(data)** — JSON ma'lumotdan
- ichki **normalize*** — maydonlarga default qiymat beradi, **http://** prefix qo'shadi, validatsiya qiladi.

internal/engine — yadro

Asosiy tiplar:

- **Engine** — asosiy ob'ekt; **New(scenario)** bilan yaratiladi, **Run(ctx, opts)**.
- **LoadProfile** (interface) — **ConcurrencyAt(t)**, **MaxConcurrency()**, **Duration()**.

Implementatsiyalari: **steady**, **ramp**, **spike**, **wave**, **constant-rps**

(**profile.go**).

- **RunOptions** — **Out**, **Quiet**, **Capture**, **Pool**, **Progress**.
- **ProgressTracker** — live progress bar.

Muhim qo'shimcha fayllar:

- **client.go** — HTTP so'rov yuborish (connection pooling, keep-alive)
- **grpc_client.go** — gRPC
- **ws_client.go** — WebSocket
- **raw_client.go** — raw TCP + **classifyNetError** (xato tasnifi)
- **streaming.go** — oqimli (streaming) javoblar, katta hajmlar
- **buffer_pool.go** — buffer qayta ishlatish (performance uchun)
- **token_bucket.go** — RPS limitlash
- **capture.go** — muvaffaqiyatsiz javoblarni ushlash
- **assert.go** — assertion tekshirish
- **vars.go** — scenario o'zgaruvchilar (variables)
- **broadcast.go** — ko'p so'rov yuborish/gate
- **progress.go** — progress bar

internal/metrics

- **Telemetry** — butun test natijasi yig'indisi (**StartSampling** real-vaqt).
- **StepStats** — bitta step bo'yicha statistika.
- **Histogram** — latency taqsimoti, **Percentile()** funksiyasi.
- **Timeline** + **TimelineSample** — har soniyalik namunani ushlaydi.

internal/report

- **Report** — standart natija strukturasi (JSON serializatsiya uchun).
- **StepReport** — bitta qadam hisoboti.
- **SLAResult** / **EvaluateSLA** — me'yor tekshiruvi.
- **Compare(current, baseline)** — regressiya aniqlash.
- **GenerateHTML** / **GenerateMarkdown** / **GeneratePDF** — pentest hisoboti.
- **LoadReport** / **SaveReport** — faylga saqlash/yoqlash.
- **pdf_writer.go** / **pdf.go** — PDF yaratish (reportlab analogi, Go'da).
- **display.go** — terminalda chiroyli panel (ANSI ranglar bilan).

- `compliance.go` — PCI-DSS / SOC2 / ISO27001 muvofiqlik baholash.

internal/replay

- `PCAPParser` — PCAP fayl o'qiydi, TCP seanslarni tashkil etadi.
- `ReplayEngine` / `ReplayWorker` — real trafikni qayta yuboradi.
- `ParseHAR` — HAR (HTTP Archive) fayl.
- `Capture` — qayta ijro uchun paket/trafik to'plami.
- `types.go` — saqlanadigan ma'lumot turlari.
- **TLS xavfsizlik:** `MinVersion TLS1.2`, `--skip-tls-verify` da ogohlantirish.

internal/scanner — xavfsizlik

- `NewScanner` ustki tuzilmasi + quyi modullar:
- `vuln_scanner.go` / `cve.go` — zaiflik/CVE
- `nvd.go` — NVD ma'lumotlar bazasi
- `owasp.go` — OWASP Top 10
- `crawler.go` — sayt sahifalarini yurib (crawl) chiqish
- `auth.go` — login orqali skanerlash (AuthSession)
- `verify.go` — topilgan zaiflikni tasdiqlash (verification)
- Natija tiplari: `TLSInfo`, `WAFInfo`, `HTTPInfo`, `ScanResult`, `CVEDetector`, `OWASPChecker`.

internal/dashboard

- `Server` — HTTP server; `NewServer()`, `ServeHTTP`.
- `EngineBridge` — engine natijalarini web'ga uzatadi.
- API: `/api/snapshot`, `/api/run`, `/api/history`, WebSocket real-vaqt.
- `index.html` — brauzer tomon (interfeys).

internal/dist (distributed)

- `proto/` — gRPC ta'rifi (`coordinator.proto` dan hosil qilingan `.pb.go`).
- `MasterWorker` — `Coordinate` bidi-streaming gRPC.
- Master yukni bo'lib tarqatadi, worker'lar bajaradi, natijani yig'adi.
- Oqim: `WorkerCommand` → `WorkerEvent` (RunProgress, RunStarted, Report, ...).

api

- `Config` / `Result` tiplari.
- `Run(ctx, cfg)` — bitta funksiya orqali to'liq test; boshqa loyihalarda

Go library sifatida ishlatish uchun.

Qaysi mavzularni mustaqil o'rganish kerak (Go asoslari)

Bu loyihani chuqur tushunish uchun ushbu Go/Golang tushunchalarini bilish

zarur — ular kodda ko'p ishlatilgan:

1. **Goroutine** — `go func(){...}()`; parallel ishlash asosi.
2. **Channel** — goroutine'lar orasida ma'lumot almashish.
3. ``sync`` — `Mutex`, `WaitGroup`, `sync.Once` (konkurent xavfsizlik).
4. **Pointer / value receiver** — metodlarda `*T` vs `T`.
5. **Interface** — `LoadProfile`, `ResponseCapture` kabi; polimorfizm.
6. **Error handling** — `errors.Is`, `errors.As`, `fmt.Errorf` (%w wrap).

7. **Context** — `context.Context`, `signal.NotifyContext` (bekor qilish).
8. ``net/http`` — HTTP server & client.
9. **JSON** — `encoding/json`, `Marshal/Unmarshal`, `omitempty`.
10. **YAML** — `gopkg.in/yaml.v3` (scenario yuklash).
11. **gRPC** — `google.golang.org/grpc`, `.proto` dan kod generatsiya.
12. **WebSocket** — `gorilla/websocket`.
13. ``flag`` — CLI argument parsing.
14. **Histogram / percentile** — o'lchov taqsimoti hisobi.
15. **CI/CD** — GitHub Actions (`.github/workflows/ci.yml`, `release.yml`).
16. **Docker** — `Dockerfile`, multistage build.
17. **Performance** — buffer pool, connection reuse (qayta ishlatish).

Muammoli joylar namunalari (suhbatda gapirish uchun)

Bular loyihani chuqur bilganingizni ko'rsatadigan haqiqiy misollar:

1. **"Connection pooling / keep-alive"** — `client.go` da TCP ulanishlarni qayta ishlatish, bu orqali 66k+ RPS'ga erishish.
1. **"Race condition (gate mode)"** — origin'dan kelgan xususiyat; bir vaqtda ko'p so'rov yuborib, raqobat holatini (race) aniqlash.
1. **"Histogram percentile"** — P99 ni hisoblash uchun `metrics/histogram.go`.
2. **"PCAP replay"** — real trafikni qayta ijro qilish; TLS decrypt uchun private key.
1. **"Security compliance"** — PCI-DSS/SOC2/ISO27001 baholash — `report/compliance.go`.
1. **"Distributed"** — master/worker gRPC orqali ko'p mashinada ko'paytirish.
2. **"Fail-fast"** — o'lik nishonlarda dastur tez (12s) xato beradi — `abi` qadamida `probe`.

Tekshiruv savollari (o'z-o'zini baholash)

Har biriga o'z so'zing bilan javob bera olsangiz — tayyorsiz:

1. `main()` nima qiladi? Qanday subcomanda'lar bor?
2. `cmdRun` da Scenario qanday yaratiladi (fayl vs --url)?
3. `engine.Run` ichida nima bo'ladi (oqim)?
4. `LoadProfile` nima va qanday tiplar bor?
5. `Telemetry` va `Histogram` nima uchun kerak?
6. `report.Build` nimani qaytaradi va u qanday ishlatiladi?
7. Replay, scanner, dashboard, dist — har birining maqsadi?
8. Konkurentlik qayerda ishlatiladi? Qanday xavfsizlik choralari bor (mutex)?
9. SLA gate qanday ishlaydi va exit code nima uchun?
10. CI/CD'da qanday testlar ishlaydi?

Xulosa

- **1-2-darajalar** — portfolio/suhbat uchun yetarli (bir necha kun).
- **3-daraja** — asosiy oqimlarni kuzatish (bir hafta).

- **4-daraja** — chuqur o'rganish (ixtiyoriy, oylab davom ettirish mumkin).

Esdan chiqmasin: butun kodni yodlash kerak emas. **Katta rasmni** bilish va kerak bo'lganda fayllardan qidirish — bu haqiqiy dasturchining usuli.

— End of document. Stress-strike study guide —